1. Logistic Regression

Penjelasan: Logistic Regression adalah algoritma yang digunakan untuk memprediksi probabilitas sebuah data masuk ke dalam kategori tertentu. Meskipun namanya mengandung kata "regresi", logistic regression sebenarnya adalah algoritma klasifikasi, khususnya untuk dua kelas (binary classification). Algoritma ini menggunakan fungsi sigmoid untuk membatasi output antara 0 dan 1, yang bisa diinterpretasikan sebagai probabilitas dari kelas yang diprediksi.

Kapan Menggunakannya:

- Ketika masalah klasifikasi Anda bersifat **biner** (dua kelas), misalnya antara "ya" dan "tidak".
- Ketika fitur atau variabel input memiliki hubungan linear dengan keluaran.
- Saat dataset relatif sederhana dan tidak terlalu besar.

Analogi: Bayangkan Anda seorang penjaga pintu sebuah acara VIP. Berdasarkan daftar tamu undangan dan data yang dimiliki (misalnya usia, status pekerjaan, atau relasi dengan penyelenggara), Anda memutuskan siapa yang boleh masuk. Logistic regression bertindak seperti sistem yang menghitung peluang setiap orang diizinkan masuk atau tidak berdasarkan data tersebut.

Contoh Kasus:

- Memprediksi apakah email adalah **spam atau bukan** berdasarkan konten email.
- Memprediksi apakah seorang nasabah akan **gagal bayar pinjaman** berdasarkan data keuangan mereka.

---

2. Decision Tree

Penjelasan: Decision Tree adalah algoritma yang memisahkan data ke dalam subset-subset yang lebih kecil berdasarkan fitur yang memberikan informasi paling relevan. Algoritma ini bekerja dengan membagi data pada node berdasarkan kriteria yang optimal, hingga akhirnya sampai ke daun (leaf) yang merepresentasikan kelas yang diprediksi.

Kapan Menggunakannya:

- Ketika data bersifat kompleks, dengan banyak interaksi antar fitur.
- Saat interpretabilitas model penting, karena decision tree memberikan visualisasi pohon yang mudah dipahami.
- Jika ada missing values (nilai yang hilang) dalam data, karena decision tree dapat menangani hal ini dengan baik.

Analogi: Bayangkan Anda sedang memilih liburan. Anda mulai dengan pertanyaan besar: "Apakah saya ingin liburan di dalam negeri atau luar negeri?" Setelah itu, Anda bisa memutuskan lagi: "Apakah saya ingin ke pantai atau pegunungan?" Setiap pertanyaan membagi pilihan Anda sampai Anda menemukan destinasi liburan yang ideal. Decision tree beroperasi dengan cara yang sama, membagi keputusan berdasarkan informasi paling relevan di setiap langkah.

Contoh Kasus:

- Memprediksi apakah seseorang akan **membeli produk** berdasarkan usia, pendapatan, dan riwayat pembelian.
- Menentukan apakah pasien menderita **penyakit tertentu** berdasarkan gejala medis.

---

3. K-Nearest Neighbor (KNN)

Penjelasan: KNN adalah algoritma non-parametrik yang melakukan klasifikasi berdasarkan data terdekat dalam ruang fitur. Ketika sebuah titik data baru muncul, algoritma melihat 'K' tetangga terdekat dari titik tersebut dan memberikan prediksi berdasarkan mayoritas kelas tetangga-tetangga tersebut.

Kapan Menggunakannya:

- Ketika data bersifat **berkelompok** atau kluster dengan distribusi yang jelas.
- Jika data memiliki dimensi rendah, karena KNN dapat menjadi lambat dengan dataset yang sangat besar.
- Saat masalahnya sederhana dan memerlukan **akurasitas** tinggi.

Analogi: Bayangkan Anda sedang mencari saran restoran. Anda mendekati tiga teman terdekat Anda dan bertanya: "Restoran mana yang mereka rekomendasikan?" Jika dua dari mereka menyarankan restoran A dan satu menyarankan restoran B, Anda kemungkinan besar akan pergi ke restoran A. Inilah cara KNN bekerja—berdasarkan pilihan tetangga terdekat.

Contoh Kasus:

- Memprediksi apakah seseorang akan **mendaftar kursus** berdasarkan usia dan pekerjaan mereka.
- Mengklasifikasikan jenis **buah** (misalnya apel atau jeruk) berdasarkan ukuran dan warna.

---

4. Naive Bayes

Penjelasan: Naive Bayes adalah algoritma klasifikasi probabilistik yang didasarkan pada teorema Bayes. Ini disebut "naive" karena membuat asumsi kuat bahwa setiap fitur saling independen, yang seringkali tidak benar dalam kenyataannya. Meski begitu, algoritma ini sering memberikan hasil yang baik.

Kapan Menggunakannya:

- Ketika Anda memiliki dataset yang besar, karena Naive Bayes sangat cepat dan efisien.
- Jika Anda bekerja dengan **data teks** atau klasifikasi dokumen (misalnya, email spam atau tidak).
- Saat fitur-fitur dianggap independen, atau interaksi antar fitur tidak terlalu relevan.

Analogi: Bayangkan Anda mencoba menebak isi kotak berdasarkan banyak petunjuk yang Anda dapatkan. Setiap petunjuk Anda anggap secara independen (misalnya, ukuran, berat, bentuk). Meskipun semua petunjuk itu mungkin sebenarnya terkait, Anda menggunakan informasi tersebut secara terpisah untuk membuat kesimpulan probabilitas tentang apa yang ada di dalam kotak. Itulah cara Naive Bayes bekerja.

Contoh Kasus:

- Memprediksi apakah sebuah **review film** positif atau negatif berdasarkan kata-kata dalam teks.
- Mendeteksi apakah dokumen adalah **spam** atau tidak berdasarkan kata-kata yang muncul di dalamnya.

---

5. Support Vector Machine (SVM)

Penjelasan: SVM adalah algoritma yang mencari **hyperplane** terbaik yang memisahkan dua kelas dalam ruang fitur. SVM berupaya memaksimalkan margin antara titik data dari kedua kelas. Jika data tidak dapat dipisahkan secara linear, SVM dapat menggunakan trik kernel untuk memetakan data ke dimensi yang lebih tinggi sehingga pemisahan menjadi mungkin.

Kapan Menggunakannya:

- Ketika data Anda bersifat **high dimensional** (banyak fitur).
- Saat Anda membutuhkan model yang tangguh dalam menangani data yang sedikit tetapi berkualitas tinggi.
- Jika masalah klasifikasi bersifat **non-linear**.

Analogi: Bayangkan Anda memisahkan dua kelompok orang yang berbeda di sebuah lapangan dengan menarik garis di antara mereka. SVM akan berusaha menemukan garis yang memisahkan kedua kelompok tersebut dengan jarak terjauh, sehingga kedua kelompok memiliki "ruang" yang cukup jauh satu sama lain. Jika tidak ada garis lurus yang bisa

memisahkan mereka, SVM bisa "membengkokkan" ruang untuk menemukan pemisahan yang ideal.

Contoh Kasus:

- Memprediksi apakah **siswa lulus atau gagal** berdasarkan skor ujian dan faktor lainnya.
- Klasifikasi **citra wajah** menjadi wajah yang dikenali atau tidak dikenali.

Rangkuman Waktu Penggunaan Algoritma Klasifikasi:

1. **Logistic Regression:** Saat masalah bersifat linear dan biner.
2. **Decision Tree:** Ketika interpretasi keputusan penting dan data memiliki banyak interaksi.
3. **KNN:** Saat data memiliki distribusi yang jelas dan membutuhkan pendekatan berbasis tetangga terdekat.
4. **Naive Bayes:** Saat bekerja dengan data teks atau probabilitas kelas dengan banyak fitur yang independen.
5. **SVM:** Ketika data memiliki banyak fitur atau pemisahan kelas yang kompleks.

1. Decision Tree Regressor

Penjelasan: Decision Tree Regressor adalah algoritma yang memprediksi nilai kontinu dengan membagi data ke dalam subset-subset berdasarkan variabel yang paling informatif. Pohon ini memecah data pada "node" yang akan memberikan prediksi terbaik hingga sampai pada daun yang mewakili nilai prediksi akhir.

Kapan Menggunakannya:

- Ketika data memiliki banyak fitur yang interaktif atau non-linear.
- Saat Anda menginginkan model yang mudah dipahami dan dijelaskan.
- Cocok untuk menangani data dengan missing values dan outliers.

Analogi: Bayangkan Anda sedang menawar harga rumah. Anda mungkin akan bertanya, "Apakah rumah ini berada di pusat kota?" Jika ya, Anda akan mulai dengan asumsi harga tinggi. Jika tidak, Anda bertanya lagi, "Berapa luas rumah tersebut?" Proses ini seperti decision tree, di mana Anda membuat keputusan berurutan untuk memperkirakan harga rumah.

Contoh Kasus:

- Memprediksi **harga rumah** berdasarkan lokasi, luas bangunan, jumlah kamar, dan usia properti.
- Memprediksi **nilai ujian** siswa berdasarkan faktor seperti jam belajar, lingkungan keluarga, dan kehadiran di sekolah.

---

2. Support Vector Regressor (SVR)

Penjelasan: Support Vector Regressor adalah turunan dari algoritma Support Vector Machine (SVM) yang dirancang untuk masalah regresi. SVR mencoba menemukan hyperplane terbaik yang mendekati sebagian besar titik data dalam ruang fitur, di mana margin kesalahan kecil tetap diperbolehkan di sekitar hyperplane.

Kapan Menggunakannya:

- Ketika data bersifat high-dimensional atau memiliki banyak fitur.
- Saat hubungan antara fitur tidak linear dan model linear tidak mencukupi.
- Cocok untuk dataset yang kecil dan berkualitas tinggi.

Analogi: Bayangkan Anda sedang mengukur garis tengah jalan di antara dua trotoar. SVR bertujuan untuk menggambar garis di tengah jalan yang "seaman mungkin" dengan menghindari outliers, sambil tetap menjaga kesalahan prediksi dalam batas yang bisa diterima.

Contoh Kasus:

- Memprediksi **permintaan energi listrik** berdasarkan cuaca, waktu, dan hari dalam seminggu.
- Meramalkan **harga saham** dengan mempertimbangkan tren historis dan faktor ekonomi.

---

3. Lasso Regression

Penjelasan: Lasso (Least Absolute Shrinkage and Selection Operator) Regression adalah varian dari regresi linear yang menambahkan penalti berdasarkan ukuran absolut dari koefisien regresi. Ini efektif untuk menghasilkan model yang lebih sederhana dengan melakukan "shrinkage" pada koefisien variabel yang kurang relevan (membuatnya nol).

Kapan Menggunakannya:

- Ketika Anda ingin melakukan **seleksi fitur** secara otomatis, karena Lasso dapat mengurangi pengaruh fitur yang tidak relevan.
- Cocok untuk dataset yang memiliki banyak fitur namun hanya beberapa yang signifikan.
- Ketika Anda membutuhkan model yang mudah diinterpretasikan.

Analogi: Bayangkan Anda memiliki banyak pilihan pakaian, tetapi ingin memilih hanya beberapa pakaian yang benar-benar penting untuk acara tertentu. Lasso bekerja seperti ini: memilih fitur paling signifikan dan mengabaikan yang tidak terlalu berpengaruh.

Contoh Kasus:

- Memprediksi **harga rumah** dengan banyak fitur seperti lokasi, ukuran, fasilitas, tetapi hanya beberapa fitur yang benar-benar signifikan.
- Meramalkan **pendapatan bisnis** berdasarkan banyak variabel seperti pemasaran, harga, atau kinerja kompetitor, di mana hanya beberapa variabel yang dominan.

---

4. Ridge Regression

Penjelasan: Ridge Regression adalah regresi linear yang menggunakan penalti berbasis kuadrat dari koefisien regresi (L2 regularization). Penalti ini membantu mengatasi masalah **multikolinearitas**, di mana ada korelasi tinggi antar fitur, dan menghindari overfitting pada dataset yang memiliki banyak fitur.

Kapan Menggunakannya:

- Ketika dataset memiliki banyak fitur yang berkorelasi satu sama lain (multikolinearitas).
- Jika Anda ingin mencegah model dari **overfitting**, terutama ketika jumlah fitur jauh lebih besar daripada jumlah sampel.

- Saat Anda tidak ingin melakukan seleksi fitur yang ketat tetapi ingin mengurangi overfitting.

Analogi: Bayangkan Anda sedang membangun jembatan di atas sungai dengan banyak tali penahan di kedua sisi. Semakin banyak tali yang digunakan, semakin kuat jembatan tersebut, tetapi juga semakin sulit untuk mengatur ketegangan tali-tali tersebut. Ridge regression bertujuan untuk menjaga keseimbangan ketegangan di semua tali agar tetap kokoh namun tidak terlalu tegang pada beberapa titik.

Contoh Kasus:

- Memprediksi **penjualan produk** berdasarkan banyak faktor (seperti harga, promosi, musim, lokasi) yang mungkin memiliki korelasi satu sama lain.
- Mengestimasi **nilai properti** berdasarkan beberapa fitur properti dan demografis yang saling berkorelasi.

---

5. Linear Regression

Penjelasan: Linear Regression adalah algoritma regresi yang paling sederhana, di mana hubungan antara variabel independen (fitur) dan variabel dependen (target) dimodelkan sebagai garis lurus. Tujuannya adalah untuk menemukan garis yang paling mendekati titik-titik data.

Kapan Menggunakannya:

- Ketika data memiliki hubungan **linear** antara fitur dan target.
- Saat model yang mudah diinterpretasikan diperlukan, dan data relatif sederhana.
- Jika Anda memiliki sedikit fitur dan dataset besar yang tidak terlalu rentan terhadap overfitting.

Analogi: Bayangkan Anda sedang memperkirakan jumlah penjualan berdasarkan harga produk. Jika Anda percaya bahwa kenaikan harga secara proporsional akan menurunkan jumlah penjualan, Anda bisa menggambar garis lurus yang mewakili tren ini. Linear regression bekerja seperti ini, mencoba menemukan hubungan linear antara variabel.

Contoh Kasus:

- Memprediksi **penjualan produk** berdasarkan harga.
- Mengestimasi **konsumsi energi** berdasarkan suhu lingkungan.

Rangkuman Waktu Penggunaan Algoritma Regresi:

1. **Decision Tree Regressor:** Saat hubungan antar fitur bersifat non-linear dan Anda menginginkan interpretasi model yang mudah.
2. **Support Vector Regressor:** Saat data bersifat high-dimensional dengan pola non-linear yang perlu dipetakan.
3. **Lasso Regression:** Ketika seleksi fitur penting dan Anda ingin mengurangi pengaruh fitur yang tidak relevan.
4. **Ridge Regression:** Ketika terdapat multikolinearitas antar fitur dan Anda ingin mencegah overfitting tanpa sepenuhnya mengabaikan fitur.
5. **Linear Regression:** Ketika hubungan antar variabel bersifat linear dan Anda membutuhkan model yang sederhana dan mudah diinterpretasikan.

1. Agglomerative Hierarchical Clustering

Penjelasan: Agglomerative Hierarchical Clustering adalah metode pengelompokan yang berbasis hierarki. Proses ini dimulai dengan setiap titik data sebagai cluster terpisah, lalu secara bertahap menggabungkan cluster terdekat hingga semua data berada dalam satu cluster besar. Algoritma ini menghasilkan dendrogram, yaitu representasi berbentuk pohon yang menunjukkan bagaimana setiap cluster terhubung.

Kapan Menggunakannya:

- Ketika Anda ingin mengetahui **struktur hierarki** dari data.
- Cocok digunakan jika Anda tidak tahu jumlah cluster yang diinginkan sebelumnya.
- Jika dataset kecil hingga sedang, karena algoritma ini dapat menjadi lambat dengan data besar.

Analogi: Bayangkan Anda sedang mengelompokkan buku-buku di perpustakaan. Anda mulai dengan mengelompokkan buku-buku berdasarkan penulis, kemudian dari penulis, Anda mengelompokkan lagi berdasarkan genre, dan akhirnya berdasarkan tahun penerbitan. Pada akhirnya, Anda mendapatkan hierarki klasifikasi yang memudahkan dalam mengelola koleksi.

Contoh Kasus:

- Pengelompokan **genetik** berdasarkan kesamaan sekuens DNA.
- Mengelompokkan **dokumen riset** berdasarkan kesamaan topik dan sub-topik.

---

2. Gaussian Mixture Model (GMM)

Penjelasan: GMM adalah metode clustering yang mengasumsikan bahwa data berasal dari kombinasi beberapa distribusi Gaussian. Algoritma ini menggunakan pendekatan probabilistik, di mana setiap titik data memiliki probabilitas tertentu untuk menjadi bagian dari masing-masing cluster. GMM cocok digunakan saat data membentuk cluster yang berbentuk elips atau tidak simetris.

Kapan Menggunakannya:

- Ketika cluster dalam data tidak berbentuk bulat sempurna atau simetris.
- Jika Anda membutuhkan pendekatan probabilistik untuk mengetahui seberapa kuat setiap titik data berada di cluster tertentu.
- Cocok untuk dataset di mana **bentuk cluster tidak jelas** atau cenderung overlapping.

Analogi: Bayangkan Anda seorang koki yang mencampur beberapa saus, dan Anda tidak tahu secara pasti dari mana asal masing-masing rasa. Anda bisa menebak bahwa setiap rasa berasal dari kombinasi beberapa bahan utama, tetapi setiap bahan memiliki kontribusi yang

berbeda-beda untuk setiap rasa yang Anda cicipi. Begitu pula dengan GMM, di mana setiap data berasal dari distribusi yang berbeda tetapi dengan probabilitas tertentu.

Contoh Kasus:

- Pengenalan **pola suara** di mana suara-suara memiliki pola yang mirip tetapi saling overlapping.
- Segmentasi **pasar** di mana setiap pelanggan dapat termasuk ke dalam beberapa segmen dengan probabilitas berbeda.

---

3. Mean Shift Clustering

Penjelasan: Mean Shift adalah algoritma clustering non-parametrik yang bekerja dengan menggeser titik data ke arah **mode** atau pusat kepadatan tertinggi. Algoritma ini mencari wilayah dengan kepadatan data yang tinggi, dan setiap titik data secara iteratif bergerak menuju pusat kepadatan terdekat. Tidak seperti K-Means, Mean Shift tidak memerlukan penentuan jumlah cluster terlebih dahulu.

Kapan Menggunakannya:

- Ketika Anda tidak tahu jumlah cluster yang tepat sebelumnya.
- Jika data memiliki cluster dengan kepadatan yang berbeda.
- Cocok untuk dataset yang memiliki **pola kepadatan yang tidak seragam**.

Analogi: Bayangkan Anda sedang mengumpulkan bebek mainan yang tersebar di kolam renang. Setiap kali Anda menggerakkan bebek, Anda mendekatkannya ke kelompok bebek terdekat lainnya sampai semua bebek berkumpul di beberapa titik pusat yang berbeda, tergantung pada kepadatan bebek di sekitar mereka. Mean Shift bekerja dengan cara yang sama, mencari pusat kepadatan data.

Contoh Kasus:

- Deteksi **objek** dalam gambar berdasarkan kepadatan piksel.
- Menganalisis **pola pergerakan** manusia dalam ruang publik atau pusat perbelanjaan berdasarkan distribusi orang di area tertentu.

---

4. DBSCAN (Density-Based Spatial Clustering of Applications with Noise)

Penjelasan: DBSCAN adalah algoritma clustering berbasis kepadatan yang dapat menemukan cluster dengan bentuk arbitrer dan dapat menangani **outlier** atau noise dengan baik. DBSCAN mengelompokkan titik-titik data yang berada di dekat satu sama lain dalam ruang kepadatan tinggi, sementara titik-titik yang berada di ruang kepadatan rendah dianggap sebagai noise.

Kapan Menggunakannya:

- Ketika data memiliki **bentuk cluster yang tidak beraturan** dan tidak berbentuk bulat.
- Saat ada **outlier** dalam data yang perlu diabaikan oleh model.
- Cocok digunakan untuk dataset yang memiliki variasi kepadatan yang signifikan.

Analogi: Bayangkan Anda berjalan di pantai dan ingin mengelompokkan batu-batu yang tersebar di pasir. Anda mungkin mengelompokkan batu-batu yang berdekatan satu sama lain menjadi satu tumpukan, sementara batu-batu yang berada jauh dari kelompok lain akan dianggap sebagai batu yang terisolasi (noise). DBSCAN berfungsi dengan cara yang sama, mengelompokkan berdasarkan kepadatan titik-titik data.

Contoh Kasus:

- Mendeteksi **wilayah penipuan** pada data transaksi berdasarkan perilaku pengguna yang tidak biasa.
- Mengidentifikasi **kelompok galaksi** di ruang angkasa dengan distribusi yang tidak beraturan.

---

5. K-Means Clustering

Penjelasan: K-Means adalah algoritma clustering yang membagi data ke dalam sejumlah cluster yang ditentukan sebelumnya (K). Algoritma ini bekerja dengan memulai dari titik pusat (centroid) acak untuk masing-masing cluster, lalu secara iteratif menyesuaikan centroid berdasarkan rata-rata titik data di masing-masing cluster. Proses ini berlanjut sampai titik pusat stabil.

Kapan Menggunakannya:

- Ketika Anda tahu jumlah cluster yang diinginkan.
- Cocok untuk data yang memiliki bentuk cluster bulat atau elips yang jelas.
- Saat Anda membutuhkan model clustering yang **sederhana dan cepat**.

Analogi: Bayangkan Anda mengelompokkan bola-bola warna-warni di sebuah lapangan berdasarkan warna. Anda memilih beberapa bola sebagai perwakilan awal (centroid) dari setiap kelompok warna, lalu mengelompokkan bola-bola lain berdasarkan warna yang paling mirip dengan bola-bola yang sudah Anda pilih sebelumnya. K-Means bekerja seperti ini, menggunakan centroid untuk mengelompokkan data berdasarkan kedekatan.

Contoh Kasus:

- Mengelompokkan **pelanggan** berdasarkan pola belanja untuk strategi pemasaran yang lebih tepat sasaran.

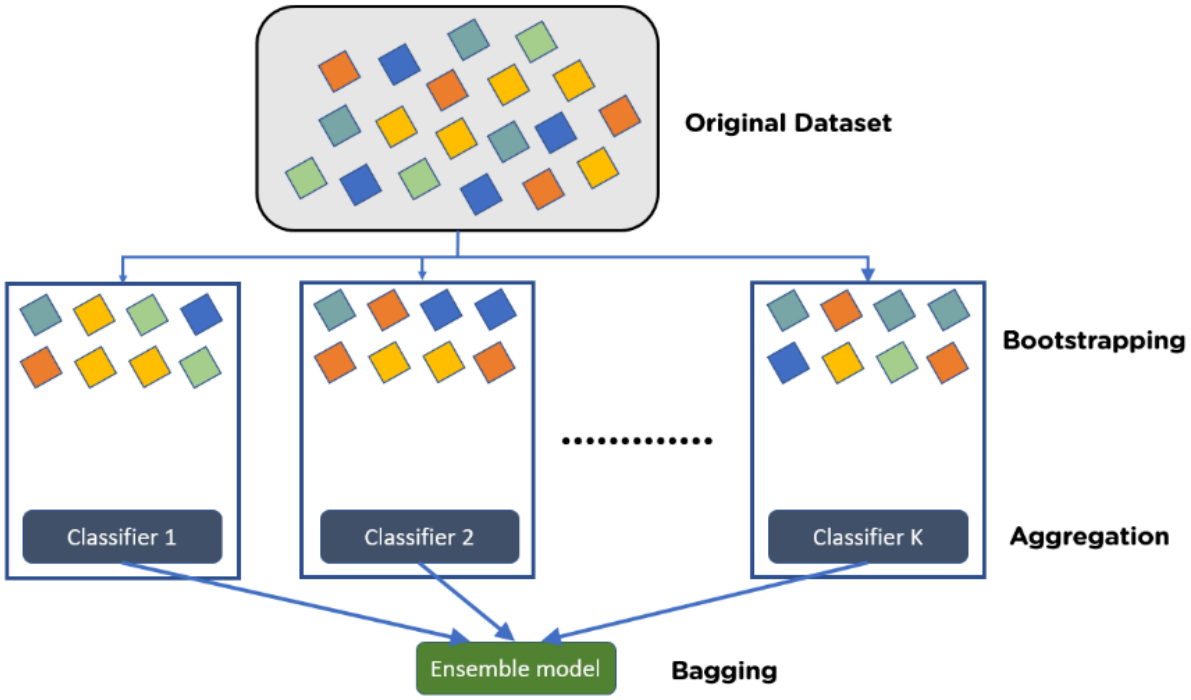
- Segmentasi **gambar** untuk menemukan area-area tertentu berdasarkan kesamaan warna atau tekstur.

Rangkuman Waktu Penggunaan Algoritma Clustering:

1. **Agglomerative Hierarchical Clustering:** Saat Anda ingin memahami **struktur hierarki** data dan tidak tahu jumlah cluster yang diinginkan.
2. **Gaussian Mixture Model (GMM):** Ketika cluster **berbentuk elips** atau tidak simetris dan Anda ingin pendekatan probabilistik.
3. **Mean Shift Clustering:** Ketika tidak diketahui jumlah cluster sebelumnya dan ada **variasi kepadatan** di data.
4. **DBSCAN:** Saat ada **cluster dengan bentuk tak beraturan** dan **outlier** yang perlu diabaikan.
5. **K-Means:** Ketika jumlah cluster diketahui dan data memiliki cluster berbentuk **bulat atau elips** yang jelas.

1. Bagging (Bootstrap Aggregating)

Bagging melibatkan pembuatan beberapa subset dari data pelatihan dengan penggantian (bootstrap) dan melatih model yang berbeda pada setiap subset. Kemudian, hasil prediksi dari model-model ini diambil rata-rata (untuk regresi) atau diambil mayoritas (untuk klasifikasi).



Sumber Gambar: Simplilearn

Baca juga : Yuk Kenali Macam-Macam Algoritma Machine Learning!

2. Random Forest

Random Forest adalah salah satu implementasi populer dari bagging. Random Forest menggunakan pohon keputusan sebagai base learner dan secara acak memilih subset dari fitur-fitur yang tersedia untuk setiap pohon serta data pelatihan, yang menghasilkan berbagai pohon keputusan yang berbeda. Kemudian, hasil dari pohon-pohon ini digabungkan untuk membuat prediksi akhir.

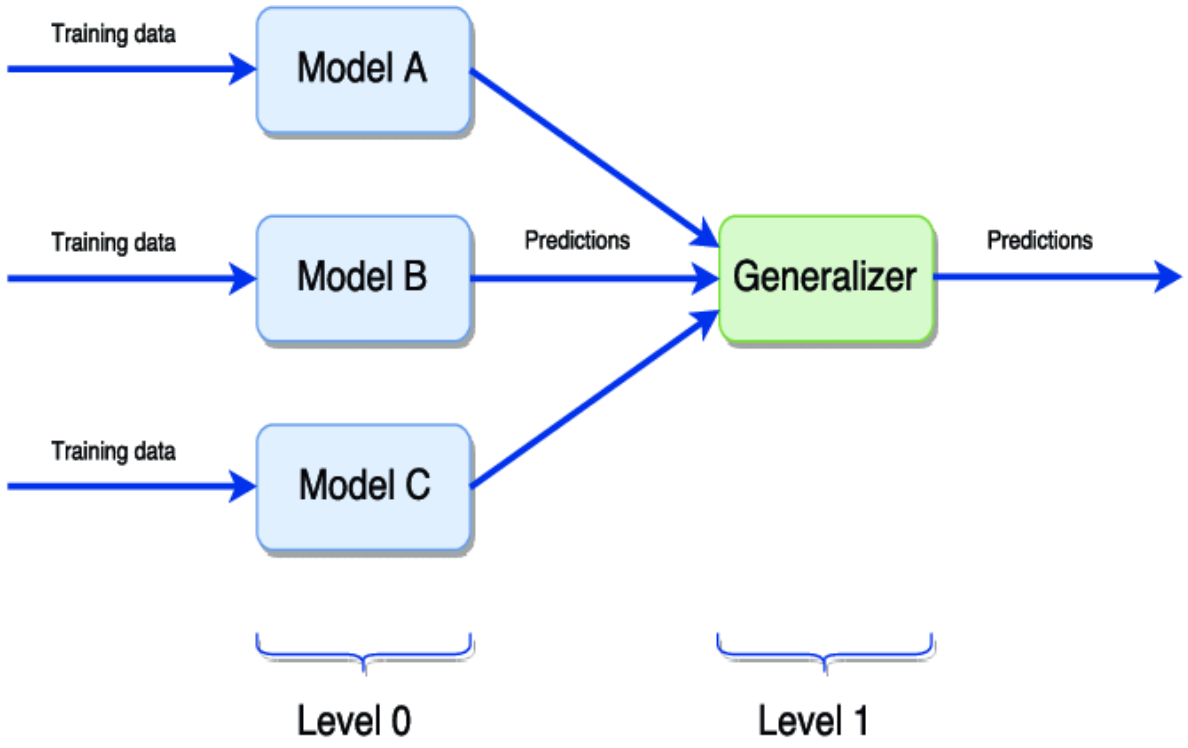
3. Boosting

Boosting adalah teknik di mana model-model lemah digabungkan secara berurutan, dengan setiap model berikutnya berusaha untuk memperbaiki kesalahan prediksi yang dilakukan oleh model sebelumnya. Contoh-contoh algoritma boosting termasuk AdaBoost, Gradient Boosting, dan XGBoost.

Baca juga : Bootcamp Machine Learning and AI for Beginner

4. Stacking

Stacking melibatkan penggabungan beberapa model berbeda dan menggunakan model yang lebih tinggi (meta-model) untuk menggabungkan hasil prediksi dari model-model tersebut. Stacking memungkinkan untuk memanfaatkan kekuatan berbagai jenis model.



Sumber Gambar: Analytics Vidhya

5. Voting

Voting adalah metode sederhana di mana beberapa model digunakan untuk membuat prediksi, dan hasilnya diambil berdasarkan mayoritas suara (untuk klasifikasi) atau rata-rata (untuk regresi).

6. Blending

Blending mirip dengan stacking, tetapi berfokus pada pemisahan data pelatihan menjadi dua set: satu untuk melatih model-base dan satu untuk melatih meta-model. Data yang digunakan untuk melatih meta-model tidak tumpang tindih dengan data yang digunakan untuk melatih model-base.

Keuntungan utama dari ensemble learning adalah peningkatan kinerja dan generalisasi model. Dengan menggabungkan model-model yang berbeda, kita dapat mengurangi overfitting dan meningkatkan ketahanan model terhadap data yang tidak terlihat sebelumnya. Namun, ensemble learning juga bisa lebih kompleks dan membutuhkan lebih banyak sumber daya komputasi daripada model tunggal.

Fungsi Aktivasi (Activation Functions)

Fungsi aktivasi adalah fungsi yang diterapkan pada output neuron dalam neural network untuk memperkenalkan **non-linearitas**. Non-linearitas penting karena banyak masalah dalam machine learning tidak memiliki hubungan linear, dan tanpa fungsi aktivasi, jaringan saraf hanya akan menjadi kombinasi linear dari input.

Berikut beberapa fungsi aktivasi yang tersedia di TensorFlow:

1. **ReLU (Rectified Linear Unit)**:
    - **Formula**: f(x)=max(0,x)f(x) = \max(0, x)f(x)=max(0,x)
    - **Penjelasan**: Fungsi ini menggantikan nilai negatif dengan nol dan mempertahankan nilai positif. ReLU adalah fungsi aktivasi paling populer di deep learning karena cepat dan mengatasi masalah vanishing gradient (gradien yang hilang).
    - **Kapan digunakan**:
        - ReLU biasanya digunakan di **hidden layers** pada deep neural networks.
        - Cocok digunakan di jaringan konvolusi (CNN) dan feedforward neural networks (FNN).
    - **Kelebihan**: Menghitung lebih cepat dan menghindari saturasi (terlalu datar) pada nilai yang besar.
    - **Kekurangan**: Masalah **dying ReLU**, di mana neuron dapat menjadi tidak aktif jika nilainya selalu negatif.
2. **Sigmoid**:
    - **Formula**: f(x)=11+e−xf(x) = \frac{1}{1 + e^{-x}}f(x)=1+e−x1
    - **Penjelasan**: Fungsi ini memetakan input ke nilai antara 0 dan 1. Nilai kecil mendekati 0 dan nilai besar mendekati 1.
    - **Kapan digunakan**:
        - Digunakan pada **output layer** untuk masalah klasifikasi **biner** (binary classification).
    - **Kelebihan**: Output mudah diinterpretasikan sebagai probabilitas.
    - **Kekurangan**: **Vanishing gradient problem**—gradien menjadi sangat kecil saat input jauh dari nol, memperlambat pembelajaran.
3. **Softmax**:
    - **Formula**: f(xi)=exi∑jexjf(x_i) = \frac{e^{x_i}}{\sum_{j} e^{x_j}}f(xi)=∑jexjexi
    - **Penjelasan**: Softmax menghitung distribusi probabilitas untuk setiap kelas. Output dari setiap kelas dijumlahkan menjadi 1, dengan setiap kelas mendapatkan probabilitas.
    - **Kapan digunakan**:
        - Biasanya digunakan di **output layer** untuk masalah **klasifikasi multi-kelas**.
    - **Kelebihan**: Output dalam bentuk distribusi probabilitas yang menjumlahkan 1, cocok untuk masalah multi-kelas.
    - **Kekurangan**: Sama dengan sigmoid, softmax bisa mengalami masalah vanishing gradient.

4. **Tanh (Hyperbolic Tangent)**:
    - **Formula**: f(x)=tanh(x)=21+e−2x−1f(x) = \tanh(x) = \frac{2}{1 + e^{-2x}} - 1f(x)=tanh(x)=1+e−2x2−1
    - **Penjelasan**: Tanh memetakan input ke nilai antara -1 dan 1. Mirip dengan sigmoid, tetapi outputnya terpusat di sekitar nol, yang membantu mengurangi masalah saturasi.
    - **Kapan digunakan**:
        - Biasanya digunakan pada **hidden layers** dalam model yang sensitif terhadap input yang berkisar antara negatif dan positif.
    - **Kelebihan**: Berpusat di sekitar nol, yang membuat gradien lebih stabil daripada sigmoid.
    - **Kekurangan**: Juga bisa mengalami masalah **vanishing gradient**.
5. **Leaky ReLU**:
    - **Formula**: f(x)=max(0.01x,x)f(x) = \max(0.01x, x)f(x)=max(0.01x,x)
    - **Penjelasan**: Modifikasi dari ReLU yang membiarkan bagian negatif dari input melewati fungsi aktivasi dengan kemiringan kecil (misal 0.01).
    - **Kapan digunakan**:
        - Alternatif ketika **ReLU** mengalami masalah **dying ReLU**.
    - **Kelebihan**: Mencegah neuron mati sepenuhnya dengan mempertahankan gradien kecil pada nilai negatif.
    - **Kekurangan**: Tidak cocok untuk semua masalah, dan memerlukan tuning untuk mendapatkan slope yang tepat.
6. **Swish**:
    - **Formula**: f(x)=x⋅sigmoid(x)f(x) = x \cdot \text{sigmoid}(x)f(x)=x⋅sigmoid(x)
    - **Penjelasan**: Fungsi aktivasi yang lebih baru yang sering digunakan di model Google seperti **EfficientNet**. Swish menawarkan kinerja yang lebih baik dalam beberapa kasus karena sifat non-monotonic-nya.
    - **Kapan digunakan**:
        - Eksploratif dalam model deep learning besar dan state-of-the-art seperti **EfficientNet**.
    - **Kelebihan**: Dapat menghasilkan kinerja lebih baik dalam beberapa masalah yang kompleks.

Optimizer adalah algoritma yang digunakan untuk meminimalkan **fungsi loss** selama proses training dengan menyesuaikan bobot (weights) dari model.

Berikut beberapa optimizer yang tersedia di TensorFlow dan penjelasannya:

1. **Stochastic Gradient Descent (SGD)**:
    - **Penjelasan**: Optimizer paling dasar yang mengupdate bobot berdasarkan gradien dari fungsi loss. Perbedaan utama dengan Gradient Descent adalah dalam SGD, bobot diperbarui setelah melihat satu sampel data (stochastic).
    - **Kapan digunakan**:
        - Untuk masalah sederhana atau ketika Anda membutuhkan kontrol yang lebih besar atas proses optimasi.
    - **Kelebihan**: Cepat pada dataset yang sangat besar karena hanya melihat satu sampel pada setiap update.
    - **Kekurangan**: Bisa menjadi **noisy** dan mungkin tidak konvergen secepat algoritma lain.
2. **Adam (Adaptive Moment Estimation)**:
    - **Penjelasan**: Adam menggabungkan keuntungan dari **Momentum** dan **RMSProp**. Ini mengombinasikan rata-rata tertimbang eksponensial dari gradien (first moment) dan gradien kuadrat (second moment).
    - **Kapan digunakan**:
        - Biasanya digunakan secara default karena bekerja dengan baik pada sebagian besar masalah deep learning, terutama ketika Anda tidak yakin optimizer mana yang paling cocok.
    - **Kelebihan**: Stabil dan cepat konvergen, cocok untuk deep learning besar.
    - **Kekurangan**: Terkadang bisa menghasilkan solusi yang lebih buruk dibandingkan optimizer lain pada masalah tertentu.
3. **RMSProp (Root Mean Square Propagation)**:
    - **Penjelasan**: Optimizer ini mempertahankan rata-rata tertimbang dari gradien kuadrat sebelumnya dan menggunakan ini untuk menyesuaikan learning rate untuk setiap parameter. RMSProp dirancang untuk bekerja lebih baik dengan masalah **online** atau **sequential**.
    - **Kapan digunakan**:
        - Cocok untuk model **recurrent neural network (RNN)** atau masalah yang memerlukan penanganan sequence data.
    - **Kelebihan**: Baik untuk masalah non-stationary seperti data sequence atau streaming.
    - **Kekurangan**: Tidak secepat Adam pada deep learning umum.
4. **Adagrad (Adaptive Gradient Algorithm)**:
    - **Penjelasan**: Optimizer yang secara otomatis menyesuaikan learning rate untuk setiap parameter selama training. Parameter yang jarang diperbarui mendapatkan learning rate yang lebih besar.
    - **Kapan digunakan**:
        - Cocok untuk data yang **sparse** (misalnya, pemrosesan teks atau data rekomendasi).

    - **Kelebihan**: Learning rate disesuaikan secara adaptif berdasarkan parameter.
    - **Kekurangan**: Learning rate cenderung menurun terlalu cepat, sehingga model bisa terjebak di minimum lokal.
5. **Adadelta**:
    - **Penjelasan**: Modifikasi dari Adagrad yang mencoba memperbaiki masalah penurunan learning rate yang terlalu cepat dengan menghilangkan pembagian kumulatif gradien yang menyebabkan penurunan tersebut.
    - **Kapan digunakan**:
        - Ketika Adagrad tidak bekerja dengan baik pada dataset Anda.
    - **Kelebihan**: Mengatasi masalah penurunan learning rate di Adagrad.
    - **Kekurangan**: Bisa lebih lambat untuk konvergen dibandingkan Adam.
6. **Momentum**:
    - **Penjelasan**: Menambahkan faktor momentum untuk menjaga laju dari pergerakan gradien. Ini menghindari fluktuasi dan membantu model untuk konvergen lebih cepat.
    - **Kapan digunakan**:
        - Cocok ketika SGD mengalami **osilasi** yang besar.
    - **Kelebihan**: Mempercepat konvergensi dan mengurangi osilasi.
    - **Kekurangan**: Memerlukan tuning yang lebih rinci dibandingkan dengan Adam.

Kapan Menggunakan Fungsi Aktivasi dan Optimizer Tertentu

1. **ReLU**: Gunakan ReLU di **hidden layers** jika Anda memiliki jaringan deep learning yang dalam untuk tugas-tugas seperti pengenalan gambar atau teks. Ini cepat dan sering kali cukup efektif.
2. **Sigmoid**: Gunakan Sigmoid di **output layer** untuk **binary classification** ketika Anda ingin output dalam bentuk probabilitas (0 atau 1).
3. **Softmax**: Gunakan Softmax di **output layer** untuk **multi-class classification**, di mana Anda memiliki lebih dari dua kelas yang harus diprediksi.
4. **Adam**: Gunakan Adam sebagai optimizer **default**. Ini bekerja baik untuk sebagian besar masalah deep learning, termasuk klasifikasi gambar, teks, dan tugas-tugas NLP.
5. **RMSProp**: Gunakan RMSProp ketika Anda bekerja dengan data **sequence** seperti time-series atau teks dalam model **RNN**.